① Activation function ← Agenda
② Gradient descent & Types of GD's
   ① SGD
   ② Adaptive Descent
   ③ ADAM
   ④ full-Batch

0th agenda ⇒ 45 mins / Revision

①

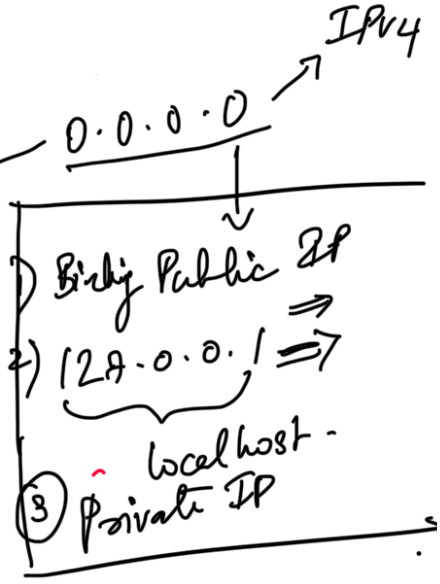
External & Internal IPs → ID / address

↳ unique address that is used to identify the server within a private network



IP/address used to identify our network on the Internet

ISP



Part I: What is deep learning?

Neural Network

"Suppose I want to predict a person's Salary based on his experience"

Q: Can We Write a Simple formula?

Yes, Salary = 20,000 + 5000(x)

Good approximate.

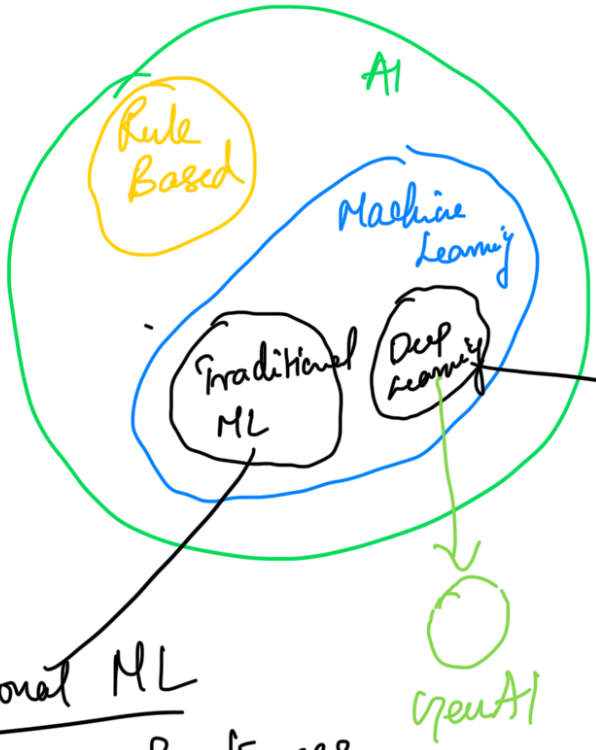
Skills, location, Experience, Company

Rule-Based AI

Rules are Written & Data follows

Machine Learning

Data arrives & Rules are based on the data



Can I do Better?

Traditional ML

① Engineered features handcrafted by us.

Deep Learning

② Automatically learns "Multiple levels" of representations by itself

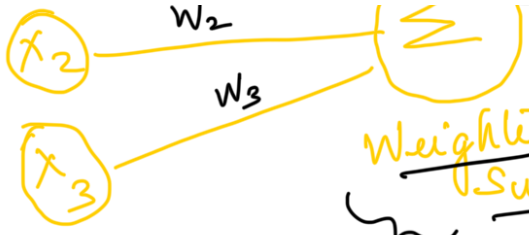
---

Part II: What is facilitating deep learning?

NEURON → Small unit of Neuron together forms a network. → NEURAL NETWORKS

→ How does this Single neuron work?





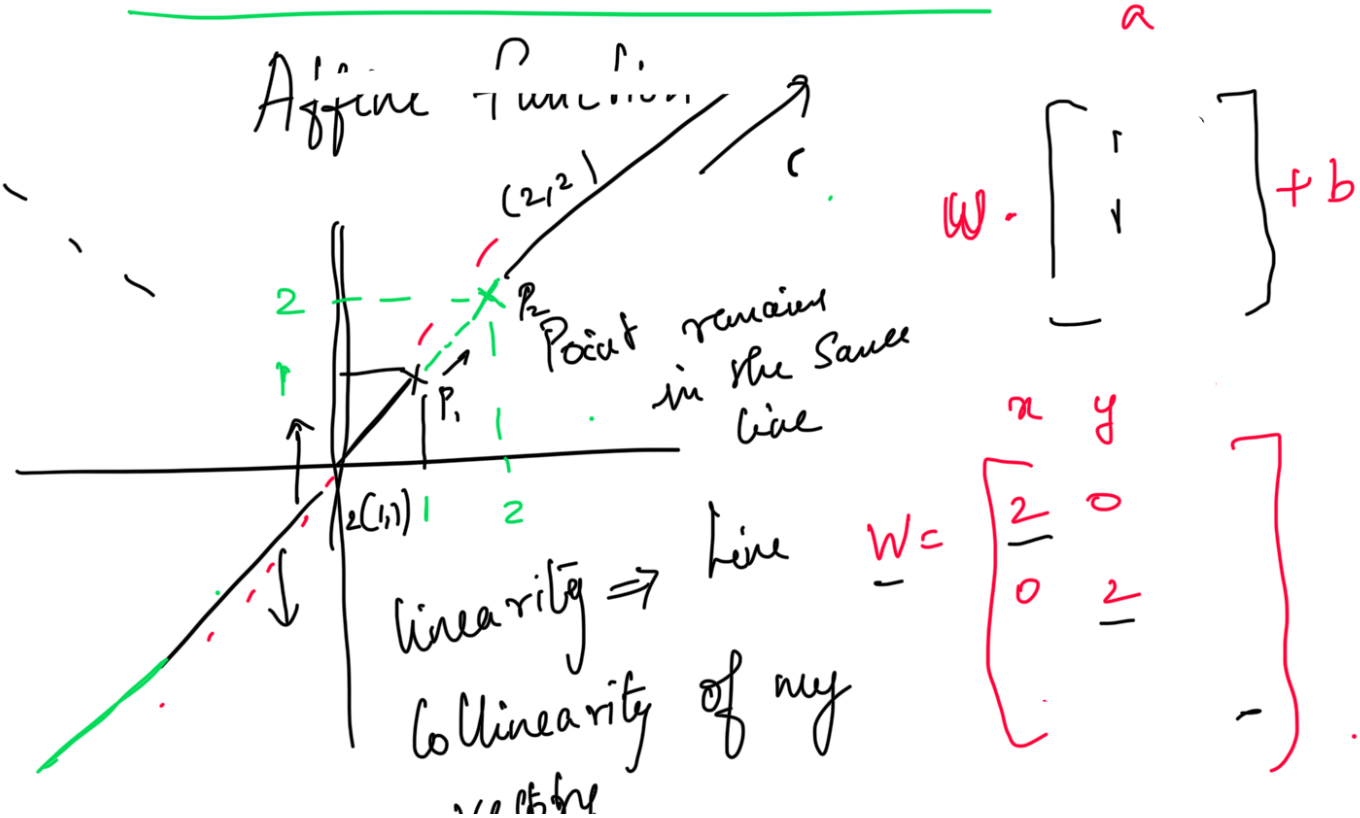
different Weights — perform a Sum

Weighted Sum $= w_1 x_1 + w_2 x_2 + w_3 x_3 + b$

(1) What is the role of weights? (In Gen Ai)
They control the influence of I/ps on the Weighted Sum.

(2) What is the role of bias?
It helps a neuron to shift its o/p / Allows the neuron to adjust its baseline or threshold.

$mx+b$ / $mx+c$ / $ax+b$ / $\beta_0+\beta_1 x$



Linear Transformation

forget the fact W is a vector

$W \cdot a + b$

$$W = \begin{bmatrix} 2 & 0 \\ 0 & 2 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

Scale → 2×2 2×1 ↑

$2 \times 1 = \begin{bmatrix} 2 \\ 2 \end{bmatrix}$

$mx + c$ / $\beta_0 + \beta_1 x$

① Points stay on the same line

② Parallel lines they stay parallel

$mx + c$ ⇒ linear function

linear Regression

What ever be the value of W & b

The line remains a line

W = Scaling, rotate, reflect

B = Translation / puts the line up & Down -

---

affine = $mx + c$ / $Wx + B$ /

↳ linear Transform

$y = mx + c$ | $Wx + b$

→ non-linear w.r.t x

$mx^2 + c$

↳ quadratic function

Partially only

m & c
are linear | non-linear

---

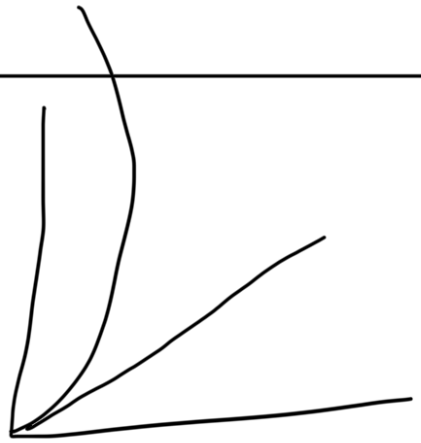
x_1 x_2 x_3 ... → x_1^2 | x_2^2 | x_3^2

$mx+c$ → $m_1 x_1^2 + m_2 x_2 + m_3 x_3$

I/Ps are non linear.

---

e^x



P III) Why do we need an activation function?

↓

Activates Non-linearity

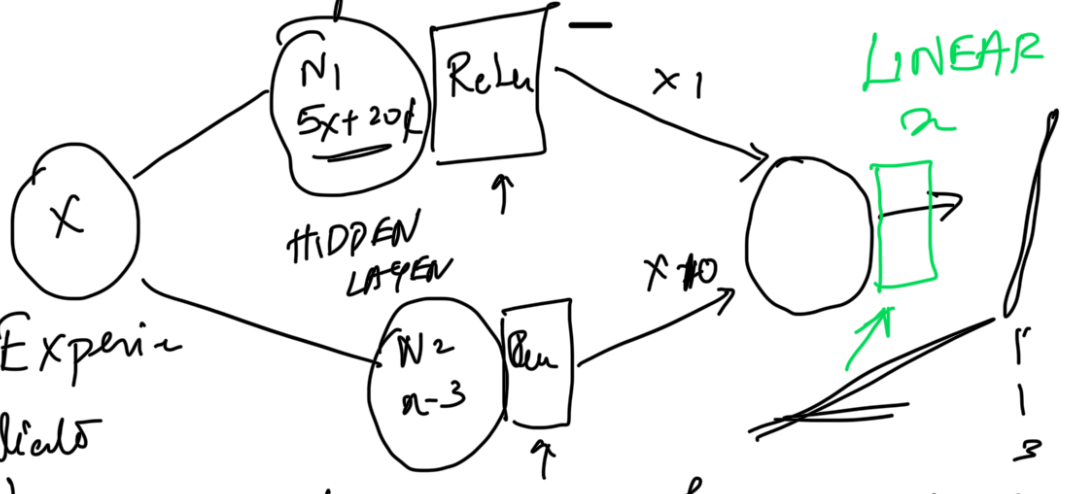
Salary = 20,000 + 5000(x) +

↓

Experience

$h_2 = 1(x) - 3 \rightarrow ReLu(\neq)$

The weighted sum combines information. The activation fn is what transforms the results



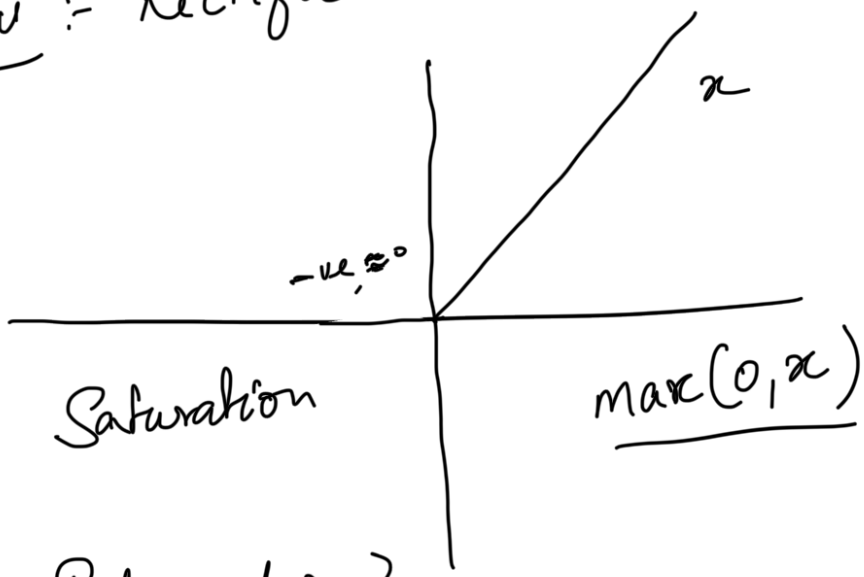
h_1 ... ... collection of neurons are

⇒ An layer with ... called what?

⇒ HIDDEN LAYER..

⇒ ACTIVATION FN.

① ReLU :- Rectified Linear Unit.



What is Relu doing?

Turn ve values Zeros. What if I don't want -ves to become 0?

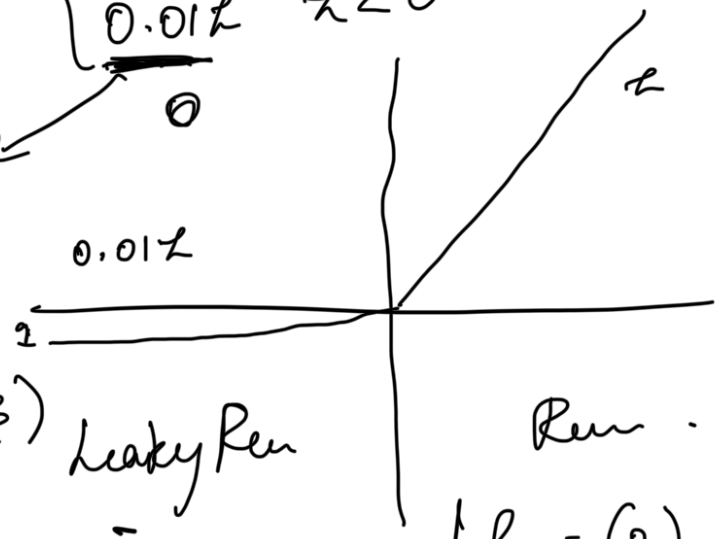
$$z = x - 3$$

⇒ Leaky Relu :-

$$f(x) = \begin{cases} z & z > 0 \\ 0.01z & z < 0 \end{cases}$$

Non-Saturating Activation fns.

fixed!!



$h_2 = \text{LeakyRelu}(x - \frac{3}{2})$ LeakyRelu Relu.

for 2 year. $h_2 = 0.01(2) = -0.01$

$\hat{S} = 30 + 10(-0.01) = 29.9$

$h_2 = (0)$

$\hat{S} = 30 + 0 = 30$

dead-Relu ⇒ gradient -ve Can become 0.

III c) what if I want probality like values?

Instead of calculatin how much experien Some one has? Imagine

WE ARE ESTIMATING ELIGIBLITY for A ₹10,000 PROMOTION BONUS.

SIGMOID

$$\sigma(z) = \frac{1}{1+e^{-z}}$$

$$h_2 = \sigma(x-3)$$

$$\hat{S} = 20 + 5x + 10(h_2)$$

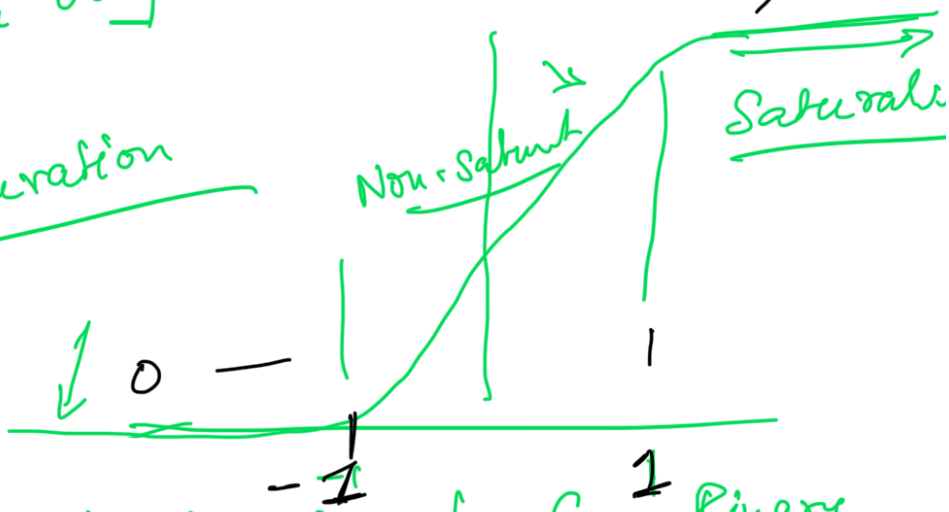
Exp	~~h_2~~ $x-3$	Sigmoid
1	-2	0.119
2	-1	0.269
3	0	0.500
4	1	0.731
5	2	0.881 ←

$$\hat{S} = 20k + 5k(x) + 10h_2$$

$$= 20 + 25 + 10(0.881)$$

$\hat{y} = 53.87$ | (0 or 1)

Cannot be in output for Continuous Prod.



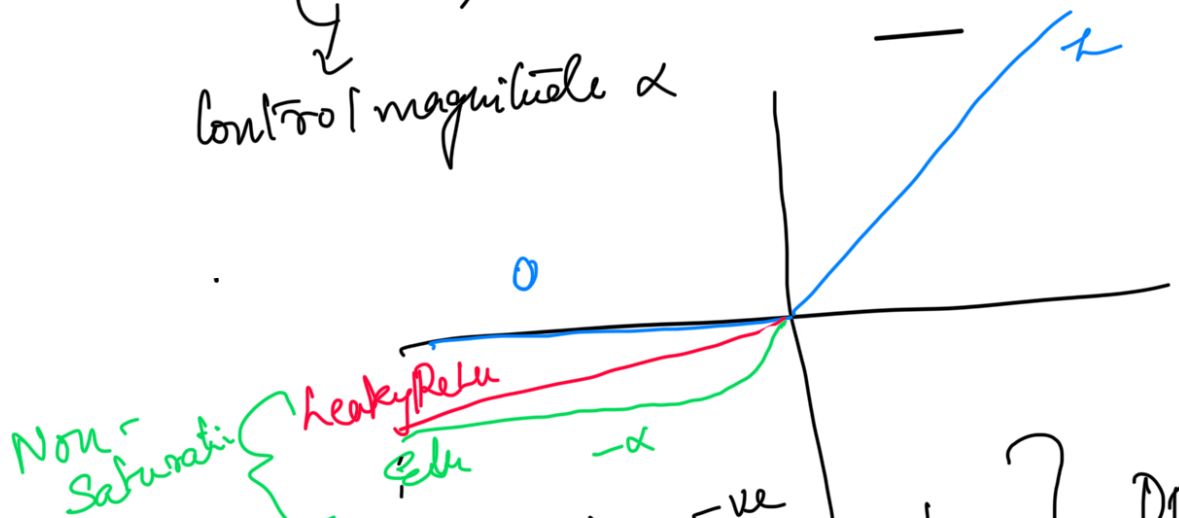
Yes it can be used in o/p only for Binary Classification.

ELU - Exponential Linear Unit

-ve = 0.01

$$ELU(x) = \begin{cases} x & x > 0 \\ \alpha(e^x - 1) & x \le 0 \end{cases}$$

↓ Control magnitude α



Ranges

Relu $[0, +\infty)$ → Partial Saturation -ve

Sigmoid $(0, 1)$ → full Saturation -ve | +ve

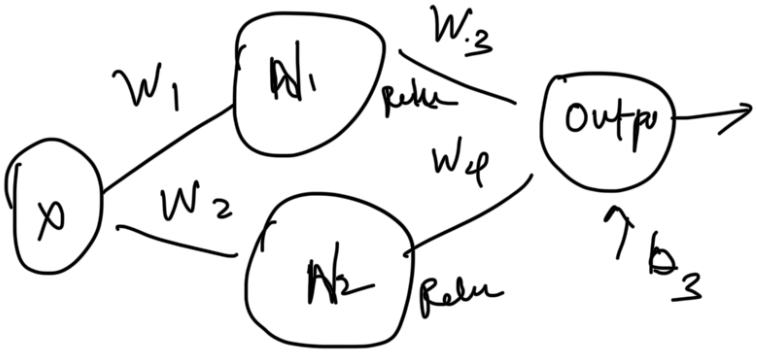
→ Non-Saturating -α

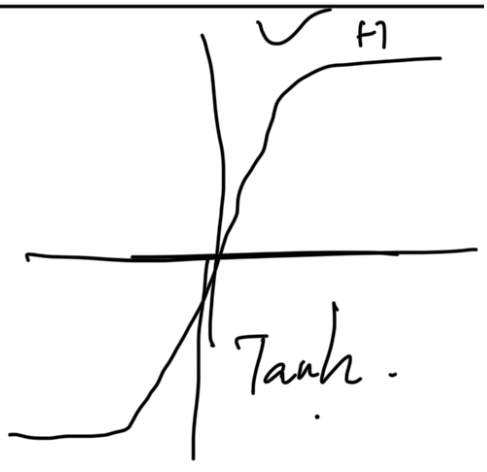
Options

Leaky Relu $(-\infty, \infty)$ → Fixed of Saturing

Elu $(-\alpha, \infty)$ → Partial

IV Optimisation





$N_1 = \text{ReLu}(W_1 x + b_1)$

$N_2 = \text{ReLu}(W_2 x + b_2)$

$\hat{y} = W_3 N_1 + W_4 N_2 + b_3$

$\theta = [W_1, b_1, W_2, b_2, W_3, W_4, b_3]$

A loss fn. determines what is the goal?

$$\mathcal{L} = \frac{1}{2}(y - \hat{y})^2$$

$$J(\theta) = \frac{1}{N} \sum_{i=1}^{N} \mathcal{L}(f_\theta(x_i), y_i)$$

Cost Fn. → Mean → sum Squares

Why a Squared loss?

① To reve -ve ~ Sign

② The Surface is convex

$$\frac{dJ}{dw_1} \quad \frac{dJ}{db_1} \quad \frac{dJ}{dw_2} \quad \cdots \quad \frac{dJ}{db_3}$$

$\begin{bmatrix} dJ/dw_1 \\ \vdots \end{bmatrix}$ all w's.

$\nabla_\theta J$ = $\begin{bmatrix} \partial J/\partial \theta_1 \\ \vdots \end{bmatrix}$ ← w's & b's

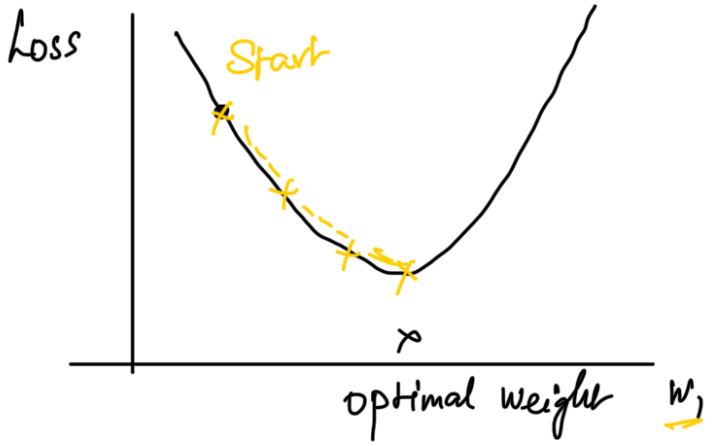
Gradient (rate of change) → Parameters $\theta = [w_1, b_1, \ldots, b_3]$

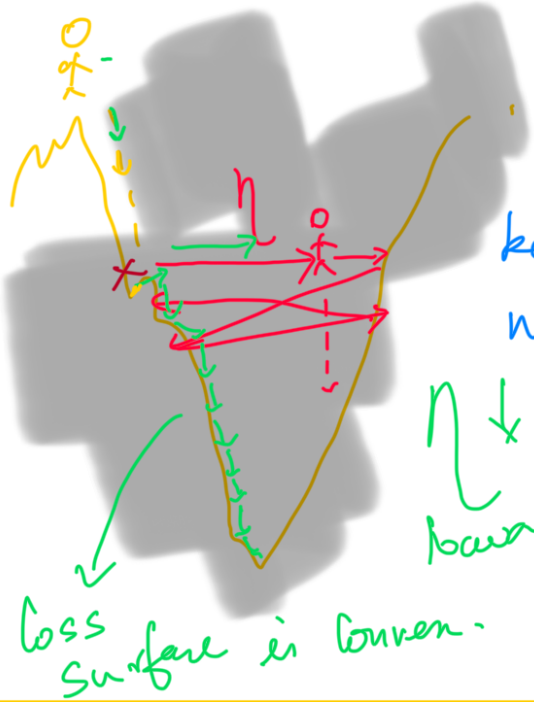
---

w_0 → Chooses η → update

$w_{new} = w_{old} - \eta \frac{\partial J}{\partial w}$

$B_{new} = B_{old} - \eta \frac{\partial J}{\partial B}$





η ↑ high I will keep shuffling not descend

η ↓ descent will be towards optim.

Loss surface is convex.

Learning Rate

Important to be low

0.001 ✓

0.00001 ✓

Is a fixed HP value.

fixed L.R · Yes will cause a

All Rows

x → ŷ

① forward pass ✓ All

② loss

③ Backpropagate find optimal values for w's / B's

④ UPDATED values

⑤ Repeat N times

$\frac{\partial J}{\partial w_1} = \frac{\partial J}{\partial y_0} \rightarrow \frac{\partial y_0}{\partial b_1}$



for all 10,000 → f.P
Calculate MSE
↓
One update

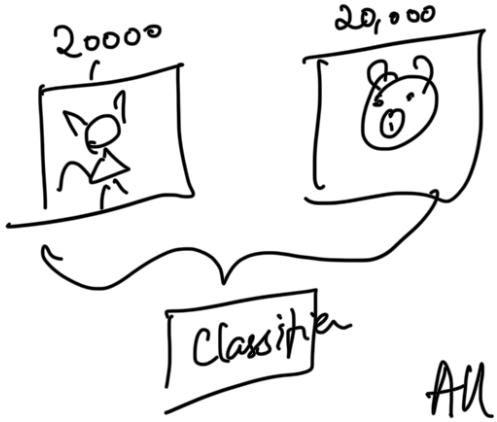
Mean for 1 update
all rows
10,000 iterations of forward pass

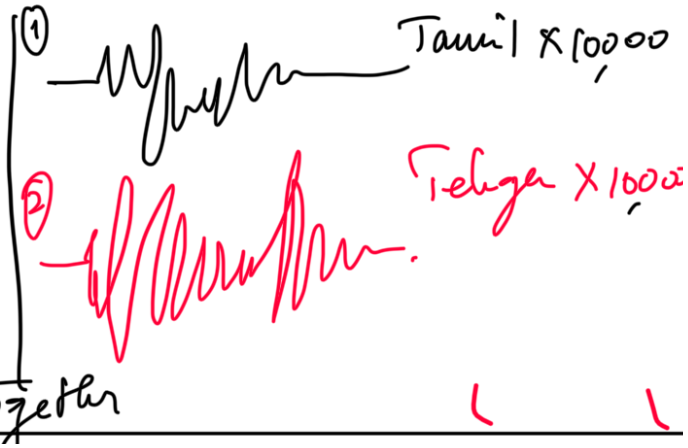
① Suppose I have 10000 Students / employees

10K emps → ① forward pass → ② MSE → ③ Gradients → ④ Parameter updates → updated Network

FULL-BATCH GRADIENT DESCENT

$\theta = (w_1, w_2)$ - 1 upda





All emp. together

ADV :-

① Gradient Represents full traing Samples
Assuming loss is an average over all Samples

② Dis adv :-

① One update will require so much Memory
② loss will be assumed Smooth

---

② Stochastic GD

⇒ Sample = 10,000 rows

⇒ 10,000 updates

One employee at a time

$$L = \frac{1}{2}(y_i - \hat{y})^2$$

ADV ① frequent
② Inexpensive

Dis adv :-
Noisy gradients

③ Mini-Batch GD

Batch Size = 1024

Samples in a Batch are randomly taken → 128, 100, 10, 5, 2

1000 | 10,000

↓

10 updates

10,000 // 10,000

few employees at a time



SGD ⇒ Diffence between SGD // BATCH GD

⇒ A fixed learning rate is not good

⇒ My next optimization can be more calculated